MYSX-256/512: A 256-bit Block Cipher with 512-bit Keys

StringManolo

2026

Abstract

I present MYSX-256/512, a 256-bit block cipher with 512-bit keys and 128 rounds. The design combines a Feistel structure with the Simon round function, a Myszkowski transposition, and an ARX key schedule. We provide the full specification, test vectors, and a security analysis covering differential, linear, and related-key cryptanalysis. Differential resistance is validated (best single-trail crosses weight 128 at r≈15, giving a margin of ~113 rounds). Linear analysis is inconclusive but the empirical evidence suggests the real attack is harder than the single-trail model predicts. The key schedule shows no exploitable weaknesses in an exhaustive audit (22 tests, coverage up to weight-6). The cipher is released as v0.1-experimental and has not been externally audited.

MYSX-256/512 — Specification

Version: 0.1 (experimental) Status: frozen. Any change requires a new version.

1. Parameters

Parameter	Value
Block size	256 bits (8 words of 32 bits)
Key size	512 bits (16 words of 32 bits)
Rounds	128
Structure	Feistel of 2 branches (128 + 128 bits)
Non-linearity	`F(x) = (rotl(x,1) ∧ rotl(x,8)) ⊕ rotl(x,2)`
Diffusion	Myszkowski transposition (keyword `MYSZKOWS`)
Rotation	`rotl(word, (r mod 31) + 1)` per word and round
Final whitening	XOR with 8 words from the key schedule

2. Notation

- `rotl(x, n)`: circular left rotation of `x` by `n` bits.
- `∧`: bitwise AND.
- `⊕`: bitwise XOR.
- `s[r]`: state at round `r` (`s[0]` = plaintext, `s[128]` = ciphertext).

- `s[r][i]`: word `i` (0..7) of the state at round `r`.
- `rk[r][i]`: subkey `i` (0..7) of round `r`.
- `rkFinal[i]`: word `i` (0..7) of the final whitening.

3. Constants

```
CONST[0..7] = [
  0x243f6a88, 0x85a308d3, 0x13198a2e, 0x03707344,
  0xa4093822, 0x299f31d0, 0x082efa98, 0xec4e6c89
]
```

These constants are the fractional digits of π (same as Blowfish).

3.1. Myszkowski permutation

Keyword: `MYSZKOWS` (8 letters, S repeated at positions 3 and 8).

Alphabetical order: K(5), M(1), O(6), S(3,8), W(7), Y(2), Z(4).

The byte permutation `MYZ_PERM[0..31]` is constructed as:

```
MYZ_PERM = [
   4,12,20,28,                 # K (column 4)
   0, 8,16,24,                 # M (column 0)
   5,13,21,29,                 # O (column 5)
   2, 7,10,15,18,23,26,31,     # S,S (columns 2 and 7, read row-wise)
   6,14,22,30,                 # W (column 6)
   1, 9,17,25,                 # Y (column 1)
   3,11,19,27                  # Z (column 3)
]
```

The inverse permutation `MYZ_INV` satisfies `MYZ_INV[MYZ_PERM[i]] = i`.

4. Key schedule

4.1. Initialization

Input: key `K[0..63]` (64 bytes).

Initial state `s[0..15]` (16 words of 32 bits):

```
s[i] = big_endian_word(K[4i .. 4i+3])
```

4.2. Expansion

For `r = 0 .. 127`:

```
spread = (r + 1) * 0x9E3779B9  (mod 2^32)

# Round subkeys
for i in 0..7:
    rk[r][i] = s[i] ⊕ s[(i+5) mod 16] ⊕ s[(i+11) mod 16]
```

```
                ⊕ CONST[(r+i) mod 8] ⊕ spread

# For i = 0..3: fold fk into rk
for i in 0..3:
    fk = s[(i+8) mod 16] ⊕ CONST[(r+i+3) mod 8]
    rk[r][i] = rk[r][i] ⊕ fk

# State update (Feistel 8+8, bijective)
L = s[0..7]
R = s[8..15]
for i in 0..7:
    a = R[i]
    b = R[(i+1) mod 8]
    c = R[(i+3) mod 8]
    F = (rotl((a + b) mod 2^32, 7) ⊕ rotl(c, 13) ⊕ CONST[i] ⊕ spread)
    s[i]     = R[i]
    s[i + 8] = L[i] ⊕ F
```

4.3. Final whitening

After the 128 rounds:

```
for i in 0..7:
    rkFinal[i] = s[i] ⊕ s[(i+8) mod 16] ⊕ CONST[(128+i) mod 8]
```

5. Round function

Each round `r` transforms state `s[r]` into `s[r+1]` in four steps:

5.1. AddRoundKey

```
for i in 0..7:
    s[r][i] = s[r][i] ⊕ rk[r][i]
```

5.2. Feistel per pair

For `p = 0..3`:

```
L = s[r][p]
R = s[r][p + 4]
s_mid[p]     = R
s_mid[p + 4] = L ⊕ F(R)
```

where `F(x) = (rotl(x,1) ∧ rotl(x,8)) ⊕ rotl(x,2)`.

5.3. Myszkowski transposition

Convert `s_mid[0..7]` to bytes (big-endian per word):

```
bytes[4i .. 4i+3] = big_endian(s_mid[i])
```

Apply the permutation:

```
permuted[j] = bytes[MYZ_PERM[j]]  for j = 0..31
```

Reconstruct words:

```
s_perm[i] = big_endian_word(permuted[4i .. 4i+3])
```

5.4. Word rotation

```
shift = (r mod 31) + 1
for i in 0..7:
    s[r+1][i] = rotl(s_perm[i], shift)
```

6. Encryption

```
s[0] = plaintext  (as 8 big-endian words)
for r = 0..127:
    apply round r (section 5)
for i = 0..7:
    s[128][i] = s[128][i] ⊕ rkFinal[i]
ciphertext = s[128]  (as 32 big-endian bytes)
```

7. Decryption

Inverse of encryption:

```
s[128] = ciphertext
for i = 0..7:
    s[128][i] = s[128][i] ⊕ rkFinal[i]
for r = 127..0:
    apply inverse round r (inverse of section 5, in reverse order)
plaintext = s[0]
```

The inverse round consists of:

1. Inverse rotation: `rotl(word, 32 - shift)`.
2. Inverse transposition: `bytes[MYZ_PERM[j]] = permuted[j]`.
3. Inverse Feistel: `R = newL`, `L = newR ⊕ F(R)`.
4. XOR with `rk[r][i]`.

8. Mode of operation (CTR)

32-byte counter:

```
counter[0..15]  = nonce (16 bytes)
counter[16..31] = 0  (128 bits, big-endian)
```

For each 32-byte plaintext block:

```
keystream = MYSX_encrypt(counter)
ciphertext_block = plaintext_block ⊕ keystream
```

```
counter[16..31] += 1  (big-endian)
```

9. AEAD (encrypt-then-MAC)

Packet: `VERSION(1) ‖ NONCE(16) ‖ CT(n) ‖ TAG(32)`

MAC input: `VERSION ‖ NONCE ‖ AAD_LEN(4 BE) ‖ AAD ‖ CT_LEN(4 BE) ‖ CT`

TAG: `HMAC-SHA256(macKey, MAC input)`

Verification: the decryptor verifies the TAG before decrypting.

10. Key derivation

10.1. From a random secret (HKDF-SHA256)

```
encKey = HKDF-SHA256(secret, salt, info = context ‖ ":enc", L = 64)
macKey = HKDF-SHA256(secret, salt, info = context ‖ ":mac", L = 32)
```

10.2. From a password (PBKDF2-SHA256 + HKDF)

```
prk    = PBKDF2-SHA256(password, salt, iterations = 600000, L = 32)
encKey = HKDF-SHA256(prk, salt, info = context ‖ ":enc", L = 64)
macKey = HKDF-SHA256(prk, salt, info = context ‖ ":mac", L = 32)
```

11. Test vectors

See KAT.json.

12. References

- Simon (NSA, 2013): function `F`.
- Myszkowski (1902): transposition with repeated-letter keyword.
- Blowfish (Schneier, 1993): π constants.
- HKDF (RFC 5869), PBKDF2 (RFC 8018), HMAC (RFC 2104).

13. Warning

MYSX **has not been externally audited**. The security analysis in SECURITY.md documents what has been tested and what has not. **Do not use in production.**

MYSX-256/512 — Security Status (v0.1-experimental)

Proprietary cipher. NOT externally audited. DO NOT use for sensitive data.
For production: AES-256-GCM or XChaCha20-Poly1305.

Freeze tag: `v0.1-experimental`. Reference KAT: `KAT.json` (digest `5c9ca63d19e0f385eb0142bbe0337`

Summary

Attack family	Status	Evidence
Differential (MILP)	**Resistant**	Weight 114 at r=14, threshold r≈15, margin ~113. Empirically validated at r=4.
Linear (MILP)	**Inconclusive, not alarming**	Single-trail: artifacts (do not verify). Strict: broken model. Empirical: no detectable correlation in weight-8 trails.
Related-key	**No exploitable weaknesses**	22 tests PASS: no persistent `Δrk[r]=0`, beam search score 139, no weak keys. Slow diffusion r=1–4 (structural).
Integral / impossible / boomerang / algebraic	Pending	—
Grover (quantum)	Infeasible	2^{256} iterations
Classical brute force	Infeasible	2^{512} operations
Block collisions	Infeasible	2^{128} blocks (256-bit block)

Short verdict: differential resistance is solid; linear is not demonstrated but the evidence is not alarming; the key schedule has no exploitable weaknesses. The cipher is frozen and must not be used in production without external review.

---

1. Differential cryptanalysis (MILP)

1.1. Model

- Tool: `mysx_milp.py` (PuLP + CBC/HiGHS).
- AND model: `a = Δx ∨ Δy` (upper bound; overestimates cost, never underestimates).
- Input difference: single-bit, worst case among all 256 (full sweep at r=5–8).
- Solver: CBC with `gapRel=0.2`, `timeLimit=300`.

1.2. Results

Rounds	Worst input	Weight	Single-trail prob.
5	w1b24	2	2^{-2}
6	w1b24	6	2^{-6}
7	w2b30	10	2^{-10}
8	w2b30	18	2^{-18}
12	w2b30	68	2^{-68}
14	w2b30	114	2^{-114}

Growth r=12→14: **+23 bits/round**, accelerating. Practical threshold (weight 128): crossed at **r≈15**. Margin with 128 rounds: **~113 rounds**.

1.3. Empirical verification

- Trail at r=4, w0b0, N=2,000 samples.
- Match rate 0.5–1.15% → prob ≈ 2^{-7}.
- Consistent with the model (2^{-8}) and with **differential clustering** (factor ~2 over single-trail).

1.4. Limitations

- Single-trail, single-input. Does not model clustering at many rounds.
- No related-key.
- The model is an upper bound: high weights are not conclusive for security; low weights indicate weakness. Here the weights are high and growing.
- `gapRel=0.2` leaves a 20% margin on the reported weight (real may be ~20% lower). The conclusion does not change.

---

2. Linear cryptanalysis (MILP) — INCONCLUSIVE

Three models were tried. None is conclusive. The empirical evidence suggests the real attack is **harder** than the models predict, but it is not a proof.

2.1. Hybrid model (`act = α ∨ β ∨ γ, without γ ≥ α`)

Results: trails with growing weights (weight 8 at r=3, 18 at r=4, 86 at r=6, 434 at r=10).

Empirical verification: - Trail at r=4 (weight 8, expected corr $2^{-8} = 0.0039$). - 8 runs with N=2,000,000 each. - Observed correlation: mean −0.0004, ranging −0.0014 to +0.0005. - **Not significant** (≈ noise).

Diagnosis: the model allowed `(α,β,γ) = (1,0,0)`, which has correlation 0 in the real AND (not 1/2). The trails found were artifacts.

2.2. Strict model (`x = y = z = w` in XOR)

Results: **INFEASIBLE** in all rounds 1–10 for mask w0b0.

Diagnosis: the model is too restrictive. It forbids valid trails with correlation 1 (e.g., the one at r=1 with weight 0), because in the XOR `w = x ⊕ y ⊕ z` correlation 1 requires only an even number of active masks, not all masks being equal.

2.3. Relaxed model with correct AND (`x+y+z-w=2u, γ = α OR β`)

Results: growing weights (8 at r=3, 18 at r=4, 86 at r=6, 434 at r=10).

Empirical verification: - Trail at r=3 (weight 8, expected corr $2^{-8} = 0.0039$). - N=5,000,000. - Observed correlation: -0.000238, at **0.53σ** from zero. - **Not significant**.

Diagnosis: model artifact or **destructive clustering** (multiple trails with opposite signs cancelling each other). In either case, the real attack is **harder** than single-trail suggests.

2.4. Conclusion of linear analysis

Single-trail linear analysis is inconclusive. The empirical evidence suggests the real correlation is **much lower** than the models predict. This is **good for security**: the real linear attack is harder than the model suggests. But it is not a proof.

A rigorous linear analysis would require:

1. **Sign tracking** (±1/2 per AND), which the linear MILP does not capture.
2. **Linear hull analysis** (clustering of multiple trails with the same input/output mask).
3. **More sophisticated SAT/MILP model** with XOR correlations < 1.

That is out of scope for this work.

---

3. Key schedule analysis

Tools: `keyschedule_test.js` (basic audit) and `keyschedule_test_extended.js` (exhaustive audit, 22 tests).

3.1. Coverage

Test	Coverage
Verification vs prod	1 key, 128 rounds
Bijectivity	128 rounds, full reconstruction
Null space rk[0]	255 patterns × 3 modes
Null-space propagation	765 trials
Weight-1	512 bits exhaustive

Test	Coverage
Weight-2	130,816 pairs exhaustive
Weight-3 “same Δ”	560 combos × 3 Δ = 1,680 trials
Beam search	Up to weight-6, beam 50
Multi-base	5 distinct base keys
Structured	20 patterns (symmetric, periodic, block, palindromic)
Cycles / fixed points	10,000 keys
Weak-key scan	25 patterns
Influence matrix	512 bits → rk[64]

3.2. Results

Metric	Value
Persistent `Δrk[r]=0` (r≥1)	**0** in all tests
Beam search (worst `max_r weight`)	139 (ideal 128)
Minimum weight at r≥8	103
Maximum low-weight window (<20)	3–4 rounds
Extension with threshold 40	4 rounds
Worst bit (weight-1, lowest weight at r=1)	160 (word 5, bit 0), 1 bit
Worst bit (weight-1, longest window)	1, 3 rounds with threshold 20
Best extension (threshold 40)	bit 4, 4 rounds
Worst-weight spread across 5 bases	0
Same worst bit in all bases	Yes (bit 160)
Cycles / fixed points	None (10k keys)
Weak keys	None (25 patterns)
Influence at r=64	mean 0.498, min 0.406, max 0.590

Minimum weight per round (over 512 weight-1 ΔK):

Round	min	max
1	1	26
2	6	55
3	10	107
4	27	128
5	48	149
6	81	153
8	103	150
12	106	152
16	98	154
32	107	155
64	104	151
128	105	154

3.3. Interpretation

- **No persistent `Δrk[r]=0`.** The null space of `rk[0]` (255 patterns) does not survive round 1. Propagation with `all-ones` gives ≥96 bits of difference at `rk[1]`. There is no trivial related-key distinguisher.
- **No persistent trail.** Beam search (weight ≤6, beam 50) confirms that no low-weight `ΔK` keeps low weight across all rounds. The worst `max_r weight` is 139, close to the ideal 128.
- **Slow diffusion at r=1–4.** Expected by design: the state update is a single Feistel round per cipher iteration. Diffusion completes at r=8 (minimum weight 103). Not a break.
- **Bit 160 (word 5, bit 0) is the worst bit in minimum weight.** `ΔK` with only that bit yields `Δrk[1]` with 1 bit of difference. It is **consistent across 5 distinct base keys** (spread 0), so it is structural, not noise. It is not exploitable: beam search confirms it diffuses to ~139 bits in the tail.
- **Bit 1 is the worst in window length.** It provides 3 consecutive rounds with weight <20 (r=1:2, r=2:6, r=3:17, r=4:74). It is a short window, not a persistent trail.
- **Bit 4 extends the most with threshold 40.** Window of 4 rounds (r=1..4) with weights 2, 6, 14, 36. At r=5 it jumps to 65.
- **Maximum low-weight window: 3–4 rounds.** With threshold 20, window 3; with threshold 40, window 4. No persistent trail beyond.
- **No weak keys, cycles or fixed points.** 25 patterns and 10k random keys, none pathological.

3.4. Limitations

- **Targeted MILP/SAT not performed.** The full related-key model requires modeling modular addition with carries (~650k variables for 128 rounds × 16 words × 32 bits). Not viable with CBC/HiGHS here.
- **Beam search up to weight-6.** An attacker with more resources could search weight-7+ with targeted MILP.
- **Adaptive related keys** (choosing ΔK based on the target key) not covered.
- **Biclique or algebraic attacks** on the key schedule not covered.

3.5. Key schedule verdict

No known exploitable weaknesses. Exhaustive coverage up to weight-2, weight-3 (same Δ), beam search up to weight-6, multi-base and structural analysis. Slow diffusion at r=1–4 (expected) and a structural worst bit (bit 160) that is not exploitable.

To eliminate the slow diffusion: redesign the key schedule in `v0.2` with two Feistel rounds per iteration (doubles the key schedule cost, ~10% of total). That requires full re-evaluation (differential + linear + key schedule). **Not necessary to publish v0.1-experimental.**

---

4. Generic attacks

Attack	Complexity	Verdict
Classical brute force	2^{512} operations	Infeasible
Grover (quantum)	2^{256} iterations	Infeasible with foreseeable technology
Block collisions (birthday)	2^{128} blocks	Infeasible

Note on Grover: MYSX has a 512-bit key, so Grover leaves 256 bits of effective security. With 1,000,000 logical qubits at 1 ns/iteration, the time would be $\sim 10^{60}$ years. Infeasible.

Note on the 512-bit key: HKDF-SHA256 produces at most a 256-bit pseudorandom key. If the master secret has 256 bits of entropy, the effective security of the system is 256 bits, not 512. The "key size" of MYSX is not the security level of the system.

---

5. Structural / Simon attacks

MYSX uses Simon's `F` but **is not Simon**:

- Feistel of **128-bit branches** (Simon uses `n` bits).
- **4 parallel `F`** per round (Simon uses 1).
- Myszkowski transposition (Simon does not have).
- Round-dependent word rotations (Simon does not have).
- Non-linear ARX key schedule (Simon's is linear).
- **128 rounds** (Simon128/128 has 68).

5.1. Published attacks on Simon128/128 (68 rounds)

Attack	Rounds	Complexity
Linear (hull)	49/68	$\sim 2^{127}$
Differential	46–50/68	$\sim 2^{125.7}$
Algebraic (SAT)	12–14/68	practical on PC
Integral	26/68	—

5.2. Do they apply to MYSX?

Not directly, due to the structural differences above. In particular:

- **Non-linear key schedule**: Simon has a linear key schedule, which allows recovering the master key from subkey bits via linear algebra. MYSX does not. This invalidates the key-recovery part of Simon attacks.
- **128 vs 68 rounds**: extending an attack from 49 to 128 requires correlation not to decay, and MYSX's transposition/rotations break the symmetry Simon exploits.
- **4 parallel `F`**: Simon trails (which assume 1 `F` per round) do not map to MYSX.

The MYSX-specific MILP analysis (sections 1, 2, 3) is what gives the evidence. Simon attacks are context, not direct transfer.

---

6. Analysis limitations

- **Differential**: single-trail, single-input, no related-key, no explicit clustering.
- **Linear**: single-trail inconclusive; linear hull analysis missing.
- **Key schedule**: no targeted MILP/SAT; beam search up to weight-6.
- **No integral, impossible, boomerang, algebraic.**
- **No independent public cryptanalysis.**
- **MILP models with `gapRel`** (0.2 differential, 0.3 linear). The real weight may be lower.
- **Empirical KAT verification**: 16 blocks with key `key[i]=i`. It is regression, not a security proof.
- **Functional tests**: 16/16 PASS. They cover round-trip, AEAD, negatives, nonce policy, PBKDF2, avalanche. They do not prove security.

---

7. Functional tests (reference)

`MYSX_ENABLE_TEST=1 node mysx.test.js` → **GLOBAL RESULT: PASS**.

Test	Result
KAT (16 blocks, key `key[i]=i`)	OK
CTR round-trip (8 lengths)	OK
AEAD round-trip + AAD	OK
AEAD rejects altered nonce	OK
AEAD rejects altered ciphertext	OK
AEAD rejects altered tag	OK
AEAD rejects different AAD	OK
NonceGuard without guard: rejected	OK
NonceGuard unsafeNoGuard: accepted	OK
NonceGuard operational limit	OK
HKDF correct lengths	OK
HKDF contexts separate keys	OK
PBKDF2 without salt: rejected	OK
PBKDF2 deterministic with same salt	OK
PBKDF2 different salts → different keys	OK
Plaintext avalanche	126.93/256 (ideal 128)

`CRYPTANALYSIS_STATUS: NOT_ESTABLISHED` — PASS only says the chosen tests did not detect functional failures.

---

8. Verdict

Differential solid. Linear inconclusive but not alarming. Key schedule with no exploitable weaknesses. No integral, no external review.

The cipher is **frozen at `v0.1-experimental`**. Any change to rounds, key schedule or `F` requires a new version (`v0.2-...`) and full re-evaluation from scratch.

Next steps, by value

1. **Publish specification, vectors and these results** for external review. This is what turns a proprietary design into a defensible proposal.
2. **Integral (MILP)**: third pillar of symmetric cryptanalysis. Computationally viable.
3. **Boomerang (BCT)**: complements differential. Viable.
4. **Algebraic (SAT)**: final verification, but expensive.
5. **Targeted related-key MILP**: formally closes the key schedule, but its cost (~650k variables) is not viable here.

What NOT to do

- **Do not add rounds** on intuition: it does not eliminate short trails, only lengthens them.
- **Do not declare MYSX "secure"**: differential, linear, related-key and integral are attack families, not all of them.
- **Do not use in production** until independent public cryptanalysis exists.
- **Do not present as "post-quantum"**: the key size is large, but post-quantum security has not been formally analyzed.

---

9. Reproducibility

File	Purpose
`SPEC.md`	Formal cipher specification
`KAT.json`	Test vectors (block, AEAD, KDF)
`generate_kat.py`	KAT generator from the implementation
`mysx.prod.js`	Implementation (production API)
`mysx.test.js`	Functional test suite
`mysx_milp.py`	Differential MILP model
`mysx_linear_milp.py`	Linear MILP model
`keyschedule_test.js`	Basic key schedule audit
`keyschedule_test_extended.js`	Exhaustive audit (22 tests)
`check_trail.js`	Empirical verification of the differential trail at r=4
`check_linear_trail2.js`	Empirical verification of linear trails

Fixed seed in tests (`0xC0FFEE`). MILPs are deterministic given the solver and `gapRel`. The KAT is deterministic (`digest 5c9ca63d...`).

Internal KAT digest (`KNOWN_KAT_DIGEST`, 16 blocks with `key[i]=i`): `3a8084d0b7df7eb691cfb544944`

Public KAT.json digest: `5c9ca63d19e0f385eb0142bbe0337771d32a25267469bc82c9cd8f0bed547eee`

---

10. References

- Simon (NSA, 2013): `F` function used in the round.
- Myszkowski (1902): transposition with repeated-letter keyword.
- Blowfish (Schneier, 1993): π constants (`0x243f6a88...`).
- HKDF (RFC 5869), PBKDF2 (RFC 8018), HMAC (RFC 2104), AES-GCM (NIST SP 800-38D).
- Sun et al. (2016): MILP model for linear cryptanalysis of Simon.
- Biryukov–Roy–Velichkov (CRYPTO 2015): differential cryptanalysis of Simon.
- Chen–Wang (2014): linear (hull) cryptanalysis of Simon (ePrint 2014/973).

---

11. Final warning

MYSX **has not been externally audited**. This document is the author's analysis, not a certification. No result presented here replaces an independent cryptographic review.

Do not use MYSX to protect real data. For production: **AES-256-GCM** or **XChaCha20-Poly1305**.